AuthiChain Production Monitoring Implementation

📊 Overview

This document describes the comprehensive production monitoring infrastructure implemented for AuthiChain. The system includes error tracking, health checks, structured logging, and performance monitoring.

🚨 Error Tracking with Sentry

Configuration Files Created

1. `sentry.client.config.js` - Client-side error tracking
2. `sentry.server.config.js` - Server-side error tracking
3. `sentry.edge.config.js` - Edge runtime error tracking
4. `instrumentation.ts` - Auto-initialization before app startup

Environment Variable Required

```
NEXT_PUBLIC_SENTRY_DSN=https://your-sentry-dsn@sentry.io/project-id
```

Setup Instructions

1. **Create Sentry Account**
   - Go to https://sentry.io
   - Create a new project (Next.js)
   - Copy your DSN

2. **Add to Vercel Environment Variables**
   ```bash
   # Via Vercel CLI
   vercel env add NEXT_PUBLIC_SENTRY_DSN

# Or via Vercel Dashboard
# Settings > Environment Variables > Add
```

1. **Features Enabled**
   - ✅ Automatic error capture (client & server)
   - ✅ Performance monitoring (traces)
   - ✅ Session replay (10% sample rate)
   - ✅ Release tracking (via Git SHA)
   - ✅ Sensitive data filtering
   - ✅ Error deduplication

What Gets Tracked

- Unhandled exceptions
- Promise rejections
- API errors
- Component errors (via ErrorBoundary)
- Performance metrics
- User session replays (on errors)

✅ Health Check Endpoints

Endpoints Created

1. `/api/health` - Overall System Health

Basic health check confirming API is operational.

Response Example:

```
{
  "status": "healthy",
  "timestamp": "2025-10-21T12:00:00.000Z",
  "environment": "production",
  "services": {
    "api": "operational",
    "database": "checking...",
    "stripe": "checking...",
    "nftStorage": "checking..."
  },
  "version": "abc123",
  "uptime": 3600
}
```

2. `/api/health/db` - Database Connectivity

Tests PostgreSQL database connection with a simple query.

Response Example:

```
{
  "status": "healthy",
  "timestamp": "2025-10-21T12:00:00.000Z",
  "service": "database",
  "responseTime": 45,
  "details": {
    "provider": "postgresql",
    "connected": true
  }
}
```

3. `/api/health/stripe` - Stripe API Check

Verifies Stripe API connectivity and mode (live/test).

Response Example:

```
{
  "status": "healthy",
  "timestamp": "2025-10-21T12:00:00.000Z",
  "service": "stripe",
  "responseTime": 234,
  "details": {
    "apiVersion": "2024-12-18.acacia",
    "mode": "live",
    "connected": true
  }
}
```


4. `/api/health/nft-storage` - NFT.Storage API Check

Tests NFT.Storage API connectivity.

Response Example:


```
{
  "status": "healthy",
  "timestamp": "2025-10-21T12:00:00.000Z",
  "service": "nft-storage",
  "responseTime": 156,
  "details": {
    "apiEndpoint": "https://api.nft.storage",
    "connected": true
  }
}
```


5. `/api/health/comprehensive` - All Services Check

Checks all services in parallel and returns comprehensive status.

Response Example:


```
{
  "status": "healthy",
  "timestamp": "2025-10-21T12:00:00.000Z",
  "totalResponseTime": 456,
  "services": {
    "database": {
      "status": "healthy",
      "responseTime": 45
    },
    "stripe": {
      "status": "healthy",
      "responseTime": 234
    },
    "nftStorage": {
      "status": "healthy",
      "responseTime": 156
    }
  },
  "environment": "production",
  "version": "abc123"
}
```

Status Codes

- **200** - All systems healthy
- **207** - Degraded (some services unhealthy)
- **503** - Service unavailable (critical services down)

Usage

Manual Check:

```
# Overall health
curl https://your-domain.vercel.app/api/health

# Specific service
curl https://your-domain.vercel.app/api/health/db

# Comprehensive check
curl https://your-domain.vercel.app/api/health/comprehensive
```

Automated Monitoring:
Configure your monitoring service (e.g., UptimeRobot, Pingdom) to check:
- `/api/health` every 1 minute
- `/api/health/comprehensive` every 5 minutes

📝 Structured Logging System

Logger Utility

Location: `lib/monitoring/logger.ts`

Log Levels

1. **DEBUG** - Development only, verbose logging
2. **INFO** - General informational messages
3. **WARN** - Warning conditions
4. **ERROR** - Error conditions
5. **CRITICAL** - Critical failures requiring immediate attention

Usage Examples

```
import { logger } from '@/lib/monitoring/logger';

// Basic logging
logger.info('User created account', {
  userId: '123',
  action: 'user_registration',
});

// Error logging
logger.error('Payment failed', {
  userId: '123',
  action: 'payment',
  metadata: { amount: 99.99 },
}, error);

// NFT operation
logger.logNFTOperation('mint', true, {
  userId: '123',
  metadata: { tokenId: '456' },
});

// Payment tracking
logger.logPayment('subscription', 99.99, 'USD', true, {
  userId: '123',
});

// Authentication
logger.logAuth('wallet_connect', true, {
  walletAddress: '0x123...',
});
```

Log Format

All logs are structured as JSON for easy parsing:

```
{
  "timestamp": "2025-10-21T12:00:00.000Z",
  "level": "info",
  "message": "User created account",
  "environment": "production",
  "userId": "123",
  "action": "user_registration",
  "metadata": {}
}
```

Viewing Logs in Vercel

1. Go to Vercel Dashboard
2. Select your project
3. Click “Logs” tab
4. Filter by:
   - Function (API route)
   - Status code
   - Time range
   - Search text

⚡ Performance Monitoring

Performance Monitor Utility

Location: `lib/monitoring/performance.ts`

Metrics Tracked

1. **API Call Duration**
2. **Database Query Duration**
3. **NFT Minting Duration**
4. **IPFS Upload Duration**
5. **Web Vitals** (client-side)

Usage Examples

```
import { performanceMonitor } from '@/lib/monitoring/performance';

// Start timer
const endTimer = performanceMonitor.startTimer('operation_name');
// ... perform operation ...
endTimer(); // Automatically tracks duration

// Track API call
performanceMonitor.trackAPICall('/api/nft/mint', 'POST', 456, 200);

// Track database query
performanceMonitor.trackDatabaseQuery('SELECT * FROM nfts', 23);

// Track NFT minting
performanceMonitor.trackNFTMinting(2345, true);

// Track IPFS upload
performanceMonitor.trackIPFSUpload(1024000, 3456);
```

Web Vitals Tracking

Core Web Vitals are automatically tracked:

- **LCP** (Largest Contentful Paint) - Loading performance
- **FID** (First Input Delay) - Interactivity
- **CLS** (Cumulative Layout Shift) - Visual stability
- **TTFB** (Time to First Byte) - Server response
- **FCP** (First Contentful Paint) - Initial render

Viewing Performance Metrics

In Sentry:
1. Go to Sentry Dashboard
2. Select “Performance” tab
3. View transaction summaries
4. Analyze slow transactions

Custom Metrics:

```
// Get performance summary
const summary = performanceMonitor.getMetricsSummary();
console.log(summary);
```

🛡️ Error Boundary Component

Usage

The ErrorBoundary is automatically applied to the entire app in `layout.tsx`, but you can add additional boundaries for specific sections:

```
import { ErrorBoundary } from '@/components/monitoring/ErrorBoundary';

function MyComponent() {
  return (
    <ErrorBoundary fallback={<CustomErrorUI />}>
      <YourComponent />
    </ErrorBoundary>
  );
}
```

Features

- Catches React component errors
- Sends errors to Sentry automatically
- Shows user-friendly error UI
- Provides reload button

📊 Monitoring Middleware

API Route Monitoring

Wrap API routes with monitoring:

```
import { monitoredAPIRoute } from '@/lib/monitoring/middleware';
import { NextRequest, NextResponse } from 'next/server';

export const GET = monitoredAPIRoute(async (req: NextRequest) => {
  // Your API logic here
  return NextResponse.json({ success: true });
});
```

Features

- ✅ Automatic request/response logging
- ✅ Performance tracking
- ✅ Error handling
- ✅ Request ID generation

🔧 Configuration Checklist

Environment Variables to Add

Variable	Required	Description
`NEXT_PUBLIC_SENTRY_DSN`	Yes	Sentry project DSN
`DATABASE_URL`	Yes	Already configured
`STRIPE_SECRET_KEY`	Yes	Already configured
`NFT_STORAGE_API_KEY`	Yes	Already configured

Vercel Configuration

1. **Add Sentry DSN**
   `bash`
   `vercel env add NEXT_PUBLIC_SENTRY_DSN production`

2. **Enable Experimental Features**

Already configured in `next.config.js`:
`javascript`
`experimental: {`
`instrumentationHook: true,`
`}`

1. **Redeploy**
   `bash`
   `vercel --prod`

📈 Recommended Monitoring Services

1. Sentry (Error Tracking)

- **Cost:** Free tier available (5,000 events/month)
- **Setup:** 5 minutes
- **Value:** Critical for production error tracking

2. UptimeRobot (Uptime Monitoring)

- **Cost:** Free tier (50 monitors)
- **Setup:** 2 minutes
- **Value:** Alerts when site goes down
- **Configure:** Monitor `/api/health` every 5 minutes

3. Vercel Analytics (Built-in)

- **Cost:** Free with Vercel Pro
- **Setup:** Automatic
- **Value:** Web Vitals, visitor analytics

4. LogDNA / Logtail (Log Aggregation)

- **Cost:** Free tier available
- **Setup:** 10 minutes
- **Value:** Advanced log search and analysis

🚨 Alert Configuration

Sentry Alerts

Recommended alert rules:

1. **Critical Errors**
   - Condition: Error with tag `level:critical`
   - Action: Email + Slack notification
   - Frequency: Immediately

2. **High Error Rate**
   - Condition: >10 errors in 5 minutes
   - Action: Email notification
   - Frequency: Once per hour

3. **Performance Degradation**
   - Condition: P95 response time >2s
   - Action: Email notification
   - Frequency: Once per day

Uptime Monitoring Alerts

1. **Site Down**
   - Check: `/api/health` returns 200
   - Interval: Every 1 minute
   - Alert: Email + SMS when down for 2 minutes

2. **Service Degraded**
   - Check: `/api/health/comprehensive` returns 200 or 207
   - Interval: Every 5 minutes
   - Alert: Email when degraded for 10 minutes

🔍 Troubleshooting

Sentry Not Capturing Errors

Check:

1. Is `NEXT_PUBLIC_SENTRY_DSN` set correctly?

`bash`

```
vercel env pull .env.production.local
cat .env.production.local | grep SENTRY
```

1. Is the DSN public (starts with `https://`)?

2. Check browser console for Sentry initialization errors
3. Verify Sentry project is not paused

Health Checks Failing

Database Check:

```
# Test locally
curl http://localhost:3000/api/health/db

# Check error
# Look for: "connection refused", "timeout", "authentication failed"
```

Stripe Check:

```
# Verify Stripe key is set
vercel env ls | grep STRIPE_SECRET_KEY

# Test API manually
curl https://api.stripe.com/v1/customers?limit=1 \
  -u sk_live_your_key:
```

NFT.Storage Check:

```
# Verify key is set
vercel env ls | grep NFT_STORAGE_API_KEY

# Test API manually
curl https://api.nft.storage/ \
  -H "Authorization: Bearer your_api_key"
```

Logs Not Appearing in Vercel

Check:
1. Are you using `logger.info()` etc.? (Not `console.log`)
2. Is the function deployed? (Check Vercel dashboard)
3. Is the function being invoked? (Check function logs)
4. Filter by “All Logs” (not just errors)

📊 Monitoring Dashboard Setup

Create a Monitoring Dashboard

Combine these tools for full visibility:

```
┌─────────────────────────────────────┐
│         Monitoring Dashboard        │
├─────────────────────────────────────┤
│ 1. Vercel Dashboard                 │
│    - Deployment status              │
│    - Function logs                  │
│    - Web Vitals                     │
│                                     │
│ 2. Sentry Dashboard                 │
│    - Error tracking                 │
│    - Performance monitoring         │
│    - Release health                 │
│                                     │
│ 3. UptimeRobot                      │
│    - Uptime status                  │
│    - Response time graphs           │
│    - Incident timeline              │
│                                     │
│ 4. Custom Health Dashboard          │
│    - /api/health/comprehensive      │
│    - Real-time service status       │
└─────────────────────────────────────┘
```

🎯 Quick Reference

Health Checks

```bash
# Production
curl https://your-domain.vercel.app/api/health
curl https://your-domain.vercel.app/api/health/comprehensive

# Local
curl http://localhost:3000/api/health
```

Import Monitoring

```typescript
// Logger
import { logger } from '@/lib/monitoring/logger';

// Performance
import { performanceMonitor } from '@/lib/monitoring/performance';

// Middleware
import { monitoredAPIRoute } from '@/lib/monitoring/middleware';

// Components
import { ErrorBoundary } from '@/components/monitoring/ErrorBoundary';
```

Common Operations

```
// Log an operation
logger.info('Operation completed', {
  userId: user.id,
  action: 'operation_name',
});

// Track performance
const endTimer = performanceMonitor.startTimer('operation');
// ... operation ...
endTimer();

// Wrap API route
export const GET = monitoredAPIRoute(async (req) => {
  // Your code
});
```

✅ Implementation Checklist

- [x] Sentry installed and configured
- [x] Health check endpoints created
- [x] Structured logging implemented
- [x] Performance monitoring added
- [x] Error boundaries added to layout
- [x] Web Vitals tracking enabled
- [x] Monitoring middleware created
- [ ] Sentry DSN added to Vercel
- [ ] UptimeRobot configured
- [ ] Alert rules configured
- [ ] Team notified of monitoring setup

📞 Next Steps

1. **Add Sentry DSN to Vercel**
   `bash`
   `vercel env add NEXT_PUBLIC_SENTRY_DSN production`

2. **Redeploy Application**
   `bash`
   `cd /home/ubuntu/authichain_premium/app`
   `vercel --prod`

3. **Test Health Checks**
   `bash`
   `curl https://your-domain.vercel.app/api/health/comprehensive`

4. **Set Up UptimeRobot**
   - Monitor `/api/health` every 1 minute
   - Monitor `/api/health/comprehensive` every 5 minutes

5. **Configure Sentry Alerts**
   - Critical errors → Immediate notification
   - High error rate → Hourly digest
   - Performance degradation → Daily summary

6. **Create Monitoring Dashboard**
   - Bookmark Vercel logs
   - Bookmark Sentry dashboard
   - Bookmark UptimeRobot status

🎉 Summary

AuthiChain now has enterprise-grade production monitoring:

✅ **Error Tracking** - Sentry captures all errors
✅ **Health Checks** - 5 endpoints monitoring all services
✅ **Structured Logging** - JSON logs for easy parsing
✅ **Performance Monitoring** - Track slow operations
✅ **Web Vitals** - Monitor user experience
✅ **Error Boundaries** - Graceful error handling

The platform is now ready for production with full observability! 🚀